SAP-C02 Domain 3 — Continuous Improvement for Existing Solutions

Average question, answer template and traps for tasks 3.1–3.5 · built from the 183 Domain 3 questions in your trainer · D1 = share one central thing · D2 = build one workload · **D3 = fix what already runs with the smallest change**

The average Domain 3 question

> "A company **currently runs** [app on EC2 / Lambda + RDS / CloudFront + ALB / Kinesis]. **[Symptom: errors at peak / too many DB connections / slow for far-away users / lost data / high bill / security finding]**. Which change fixes it with the **LEAST change / LEAST overhead** (or **MOST cost-effectively**)?" — 58 of 183 describe a failure, 41 say "currently/existing", 38 mention slow/latency.

The average answer has 4 parts

- **Where:** the bottleneck tier named by the symptom (DB connections, cold start, distance, single instance).
- **What:** the one managed feature built for that symptom.
- **How it survives:** add what's missing (2nd AZ, ASG, DLQ, replica, origin group), no rewrite.
- **How it's protected and paid for:** fix in place (WAF Count first, endpoints not NAT, Spot / Savings Plans for the right part).

The wording tells you which answer wins

- **LEAST change** → add a layer in front or beside (proxy, cache, queue, CloudFront, WAF); no migration.
- **LEAST overhead** → managed / serverless replacement (Transfer Family, IoT Core, Fargate, Step Functions).
- **MOST cost-effective** → stop paying for idle (Spot, Lambda, Serverless v2, scaling, endpoints, CloudFront).
- **"too many connections"** → RDS Proxy · **"first request slow"** → provisioned concurrency.
- **"far-away users slow"** → CloudFront / Transfer Acceleration / Global Accelerator.
- **"lost data at peak"** → SQS / Kinesis in front + DLQ.
- **"must not affect legitimate traffic"** → WAF in Count mode first, then Block.

The 5 fix-it checks

Check	Rule
What is the symptom?	Name the tier: DB connections, cold start, distance, throttling (429), single instance, idle cost.
Smallest fix?	Add one managed piece in front or beside it; no engine migration or rewrite unless asked.
Survives next time?	Multi-AZ, ASG + ELB health checks, DLQ, replica, origin group.
Can we see it next time?	CloudWatch agent/Logs, X-Ray, Application Signals SLOs, Synthetics canaries, AWS/Usage quota alarms.
Paid while idle?	Spot, scale in, serverless, gateway endpoint, CloudFront in front of data transfer.

Reading routine

- **Last sentence first:** MOST/LEAST word, how many answers (30 ask for TWO or THREE).
- Underline: the **symptom** (error, metric, complaint) · **what must not change** (no code change, same connection method, no downtime) · **numbers**.
- Cross out: rewrites, engine migrations, "bigger instance", raising limits, custom polling, the facts below.

Wrong almost every time: Lambda as custom glue, polling or for jobs over 15 min (70 wrong vs 34 right) · self-managed EC2 where a managed service exists (39 wrong vs 20 right) · "bigger instance" or "raise max_connections" for a connection storm · reserved concurrency to fix cold starts · caching to fix writes · read replicas for a write-heavy load · geolocation routing to reduce latency · Lambda@Edge on origin request to raise cache hits · WAF straight to Block · termination protection stopping Auto Scaling · Spot for always-on production · S3 interface endpoint when cost matters · Aurora Serverless v1 (end of life; A493 still expects it, current answer v2).

3.1 Operational excellence

Q: The current process deploys with downtime / loses logs / can't detect errors / uses long-lived keys.
A: The **built-in automation**: CodeDeploy traffic shifting, IaC import, CloudWatch agent, quota alarms, OIDC roles.

Need	Answer
Faster Lambda release + rollback	**SAM + CodeDeploy** traffic shifting + alarms
Manual VPC → IaC	**CloudFormation resource import**
Stack delete fails on non-empty bucket	**Custom resource** Lambda empties it
Template changes cause downtime	**Change sets** + test stage + blue/green
Logs lost on scale-in	**CloudWatch agent → Logs**

Need	Answer
Alert at 80% of a quota	**AWS/Usage metric alarm**
Pipeline uses long-lived keys	**OIDC provider + role**
SSH with audit trail	**Session Manager** (close port 22)
Prove recovery works	**FIS** + alarm stop condition
Lambda → DB connection storm	**RDS Proxy**

Checks: automated and reversible? · logs kept? · temporary credentials?

3.2 Improve security

Q: An existing workload has weak keys / public paths / leaked credentials / exposed endpoints.
A: Fix in place: **customer managed key / endpoint / Secrets Manager / Instance Connect / WAF Count first**.

Need	Answer
Only admins may decrypt	**Customer managed key**, admin-only policy (Macie finds data)
Upload "forbidden" with SSE-KMS	Add **kms:GenerateDataKey**
KMS request cost too high	**S3 Bucket Keys** or SSE-S3
Rotate every 180 days (AWS managed key)	New **CMK** + snapshot/restore
Same key in several Regions	**KMS multi-Region keys**

Need	Answer
On-premises → S3 privately	**Interface endpoint + private DNS**
Unique SSH per session, logged	**EC2 Instance Connect**
Least-privilege policy	**Generate from CloudTrail**
Rotating DB credentials + connections	**Secrets Manager + RDS Proxy**
Add WAF without breaking users	**Count → review logs → Block**

Checks: CMK where control is needed? · traffic off the internet? · no long-lived secrets?

3.3 Improve performance (biggest task: 69)

> **Q:** [Users / the database / Lambda / uploads / the stream] is slow or failing at peak.
> **A:** Add the **piece that removes that bottleneck**, closest to the cause.

Symptom	Fix
Too many DB connections (Lambda)	**RDS Proxy** (+ connect outside handler)
Read-heavy DB at peak	**Replicas / ElastiCache / DAX**
Session reads slow	**ElastiCache for Redis**
First request slow	**Provisioned concurrency**
Low cache hit, messy query strings	**CloudFront Function** (viewer request)
Custom error page	**CloudFront custom error / origin group + S3**
Static IPs + closest Region	**Global Accelerator**

Symptom	Fix
Large uploads from far away	**Transfer Acceleration + multipart**
Stream read throttling	**More shards + enhanced fan-out + backoff**
MQTT broker overloaded / data lost	**IoT Core** (+ Firehose / Kinesis)
One SFTP server	**Transfer Family** (same DNS)
Slow scale-out	**Warm pools**
New instances fail health checks	**Longer grace period**
Error budget / test at night	**Application Signals SLO** / **Synthetics**

Checks: which tier is the bottleneck? · smallest piece that removes it? · closest to the user?

3.4 Improve reliability

> **Q:** The existing app loses data / has one AZ / has no DR / fails silently.
> **A: Queue + DLQ**, **multi-AZ**, **native cross-Region replication + failover**, **notify on every failure**.

Need	Answer
Bursts, no data loss	**API Gateway → SQS → Lambda** + DLQ
Lambda calling Lambda times out	**Step Functions**
Messages in DLQ, no errors logged	**Scale-in protection** while processing
Single AZ	**Fargate across AZs + Multi-AZ DB**
DR without running DB instances	**Headless Aurora global DB**

Need	Answer
RPO minutes, Aurora + DynamoDB	**Add a Region** to both
S3/CloudFront failover within 1 s	**Replication + origin group**
Notify every Step Functions failure	**Catch → SNS task**
Prove AZ-loss survival	**FIS AZ power scenario**
Limit which VPCs talk	**TGW route table per attachment**

Checks: where can data be lost? · single AZ anywhere? · is the data already in the other Region?

3.5 Cost optimization

> **Q:** The bill is high because of idle instances / expiring RIs / data transfer / over-provisioned tables.
> **A: Match pricing to each usage pattern** and **stop paying for idle**.

Need	Answer
Always-on + interruptible parts	**Savings Plan always-on, Spot interruptible**
Busy a few hours a day	**Auto Scaling**; RIs for baseline only
Idle workers between jobs	**Lambda / Batch on Spot**
Spiky database	**Aurora Serverless v2**, Redshift Serverless / Concurrency Scaling
Savings Plan expiring on x86	**Graviton + new plan**
DynamoDB sized for peak	**Auto scaling** (+ DAX for hot keys)

Need	Answer
HPC file system idle most of the month	**S3 + FSx for Lustre per run**
NAT bytes to Kinesis / S3	**VPC endpoint**
Internet egress, static content	**CloudFront + S3 (OAC)**
Stop dev at night	**EventBridge schedule + Lambda** by tag
Production rightsizing	**Compute Optimizer + Trusted Advisor + Savings Plan** (no Spot)

Checks: what is idle? · commitment covers only the always-on part? · is data transfer the real cost?